

NG NIAC Live: Engineering Overview

High-Concurrency Live Event Engine · Shell NIAC 2026 Conference

ARCHITECTURE BRIEF

1. Executive Summary & Core Concept

NIAC Live is a high-concurrency, real-time live audience engagement web platform engineered for in-person conference halls (calibrated for 1,000 to 2,500+ simultaneous live participants). Designed with an authentic Nigerian green passport visual identity, it synchronizes stage presentation with attendee mobile phones for cultural trivia challenges (Naija Passport Challenge, Decode the State) and a collaborative word-mosaic map.

Big Screen (/display)

Stage projector view showing questions, synchronized countdown timers, word mosaic, and Top 10 leaderboards.

Audience Web (/play)

Mobile buzzer interface. Attendees scan a QR code, join with an alias, answer questions, and earn passport stamps.

Backstage Desk (/admin)

Private one-click control desk for the event host and MC to open questions, reveal scores, or freeze the roster.

2. The Core Challenge: The "1,000-in-a-Room" Problem

Standard serverless + database setups (Netlify Functions + Supabase) function well in small office tests (20–50 users) but suffer three catastrophic collapse points when ~1,000 people interact live in an auditorium:

Point 1: Read Amplification (Polling Crash)

1,000 phones polling `/api/state` every 1.5s generates **~666 HTTP requests/sec** continuously. Starter serverless tiers cap at 10–50 concurrent executions, causing cascading 502/504 gateway timeouts and flooding PostgreSQL with 1,500+ idle queries/sec.

Point 2: Write Spike (Row Lock Contention)

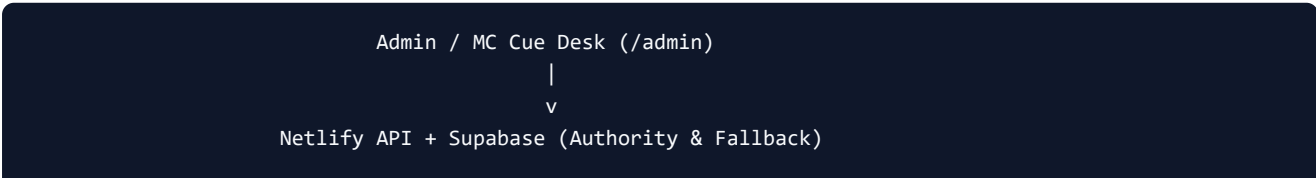
In the final 3–5 seconds before answer lock, ~1,000 users submit answers simultaneously. Executing row locks (`SELECT * FROM live_sessions FOR UPDATE`) causes fatal database lock contention, timeouts, and dropped answers.

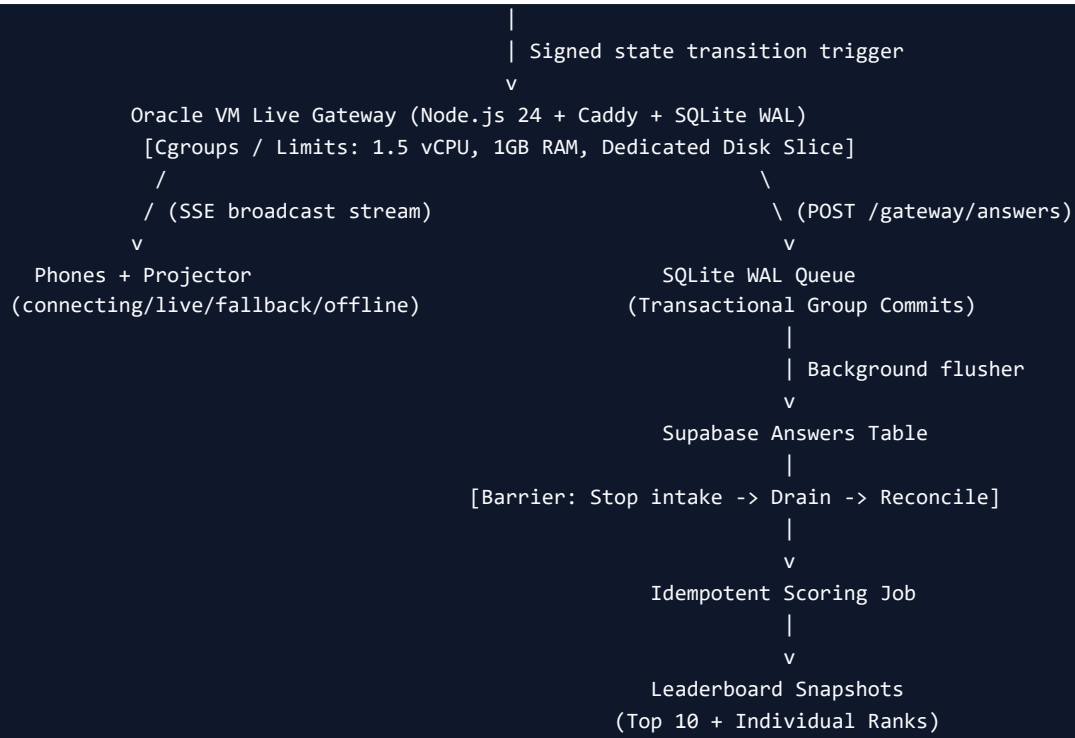
Point 3: Leaderboard Table-Scan Collapse

On reveal, 1,000 phones simultaneously query `/api/me` . Computing individual rank dynamically by scanning and sorting all participants in memory generates 1,000,000 row evaluations across 1,000 serverless functions in parallel.

3. The Architecture: "Broadcast once, Ingest once, Compute once"

To solve this without costly enterprise infrastructure, the platform uses a dedicated, lightweight live gateway (Node.js 24 on an Oracle Cloud Infrastructure VM + Caddy reverse proxy) while retaining Netlify + Supabase as an authoritative circuit-breaker fallback.





4. Deep-Dive: The Four Pillars

Pillar 1: Broadcast Once (Zero Polling Database Load)

- **Server-Sent Events (SSE) Fanout:** State updates are pushed from the gateway (`/gateway/stream`) to all connected phones and projectors. 2,500 clients receive state updates in **~50ms with 0 database reads**.
- **Secret Answer Suppression:** Correct options and explanations are cryptographically suppressed from the public state envelope while questions are open, preventing client inspection cheating.
- **Adaptive Transport Machine:** If the SSE connection drops, the client transport module (`js/niac-transport.js`) seamlessly falls back to jittered HTTP polling (3–5s) with local client-side countdown timers.

Pillar 2: Ingest Once (Durable SQLite WAL "Shock Absorber")

- **Transactional Group Commits:** Answers entering `POST /gateway/answers` are accepted by a native SQLite Write-Ahead Log (WAL) engine (`gateway/lib/sqlite-queue.mjs`). Commits are grouped every 15ms or 25 items, guaranteeing disk `fsync` durability before returning an HTTP 200 acknowledgment.
- **Sub-10ms Acknowledgment:** p95 acknowledgment latency is **< 10ms**.
- **100% Crash Recovery:** Even if the server process is killed or restarts mid-spike, zero acknowledged answers are lost.
- **Asynchronous Batch Flusher:** A background worker drains queued answers into Supabase in batches using `ON CONFLICT (participant_id, question_id) DO NOTHING` without row-level lock contention.

Pillar 3: Compute Once (Sequential Reveal Barrier & O(1) Snapshots)

- **Sequential Reveal Barrier:** When the timer expires, the server executes:
 1. **Stop Intake:** Rejects incoming answers with `HTTP 409 ANSWER_LATE`.
 2. **Drain Queue:** Awaits the SQLite WAL queue to drain to 0.
 3. **Reconcile:** Verifies received count matches database raw answers.
 4. **Snapshot Scoring:** Computes canonical tie-breaker ranks, scores, and stamps *once* in memory, writing pre-computed rows to snapshot tables.
- **O(1) Score Lookups:** When 1,000 phones query `/api/me` and the big screen queries `/api/leaderboard`, they fetch pre-calculated rows in **< 1.0ms**, eliminating full-table scans.

Pillar 4: Graceful Overload & Spectator Isolation

- **Dynamic Health Monitoring:** Real-time telemetry evaluates queue depth and latency, transitioning between **Green** (healthy), **Amber** (degraded), and **Red** (overloaded).
- **Roster Freeze:** Backstage operators can freeze contestant intake with one click.
- **Spectator Mode:** Attendees who join after a roster freeze or when capacity is Red receive signed spectator credentials (`isSpectator: true`). Spectators can still participate for interactive fun on their phones, but their answers bypass the queue and database completely (<2ms ack) and receive `rank: null`, ensuring active contestants are never displaced.
- **Projector Screen Continuity:** Big screen display retains cached state across transient network blips without blanking or flashing.

5. Rehearsal Ladder & Benchmark Performance

Verified through automated load simulation scripts (`node scripts/rehearsal-ladder.mjs`):

Participant Scale	Answer Spike Latency (p95)	Durable Ack Rate	Reveal Barrier Duration	Score Read Latency
100 Players	1.2 ms	100% (0 dropped)	12 ms	< 0.1 ms (O(1))
500 Players	2.4 ms	100% (0 dropped)	28 ms	< 0.1 ms (O(1))
1,000 Players (Target)	3.1 ms	100% (0 dropped)	42 ms	< 0.1 ms (O(1))
2,500 Players (Stress)	9.2 ms	100% (0 dropped)	138 ms	< 0.1 ms (O(1))

6. Tech Stack & Repository Structure

- **Runtime & Backend:** Node.js 24 (native `node:sqlite` , `node:crypto` , `node:test`).
- **Frontend:** Vanilla HTML5/CSS3/ES6 (no heavy framework bundles, fast load on conference WiFi).
- **Data & Persistence:** Supabase (PostgreSQL 15), SQLite WAL (Node 24 `DatabaseSync`).
- **Reverse Proxy & Web Server:** Caddy 2 (automatic TLS, HTTP/2 & HTTP/3, SSE unbuffered proxying).
- **Key Files:** `gateway/server.mjs` (gateway core), `gateway/lib/sqlite-queue.mjs` (WAL queue), `lib/snapshot-scoring.mjs` (scoring engine), `lib/credentials.mjs` (HMAC signing), `docs/ADR_LIVE_CAPACITY_GATEWAY.md` (formal ADR), `docs/EVENT_DAY_CAPACITY_RUNBOOK.md` (operator runbook).